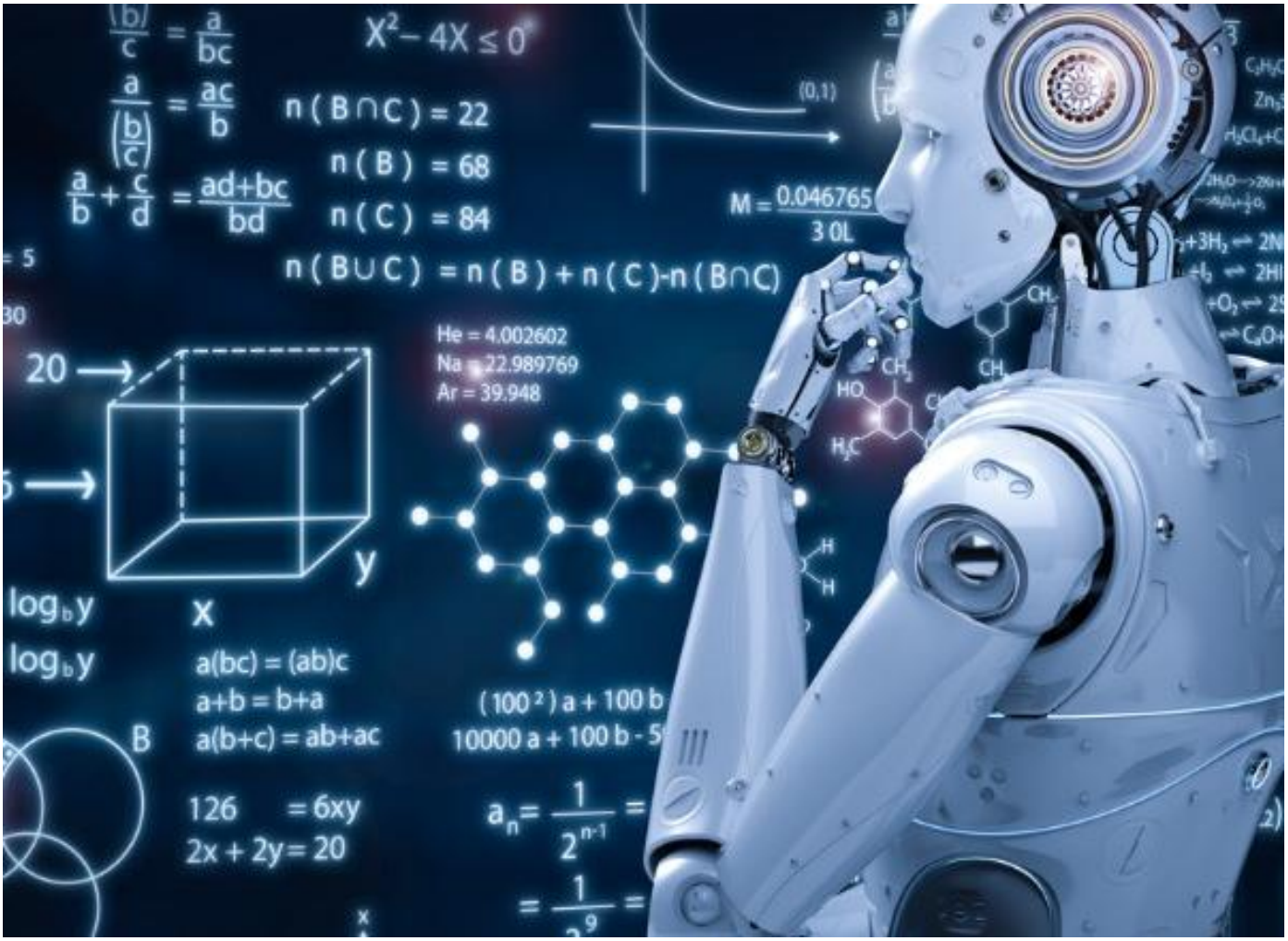


MACHINE LEARNING

MODULE 4.2

Classification

- Classification is a **supervised learning** technique used to predict a categorical label (class) for a given input based on past labeled data.

Following are the examples of cases where the data analysis task is Classification:

- A bank loan officer wants to analyze the data in order to know which customers (loan applicants) are risky or which are safe.
- A marketing manager at a company needs to analyze a customer with a given profile, who will buy a new computer.

➔In both of the above examples, a model or classifier is constructed to predict the categorical labels. These labels are risky or safe for loan application data and yes or no for marketing data.

➔In each of these examples, the data analysis task is classification, where a model or classifier is constructed to predict class (categorical) labels, such as "safe" or "risky" for the loan application data; "yes" or "no" for the marketing data. These categories can be represented by discrete values, where the ordering among values has no meaning.

Prediction

- Suppose that the marketing manager wants to predict how much a given customer will spend during a sale at AllElectronics.
- This data analysis task is an example of numeric

prediction, where the model constructed predicts a continuous-valued function, or ordered value, as opposed to a class label.

- This model is a predictor. Regression analysis is a statistical methodology that is most often used for numeric prediction; hence the two terms tend to be used synonymously, although other methods for numeric prediction exist.

Supervised Learning

- Supervised learning, as the name indicates, has the presence of a supervisor as a teacher.
- Basically supervised learning is when we teach or train the machine using data that is well labeled.
- Which means some data is already tagged with the correct answer.
- After that, the machine is provided with a new set of examples(data) so that the supervised learning algorithm analyzes the training data(set of training examples) and produces a correct outcome from labeled data.

Classification

- Naive Bayes Classifiers
- K-NN (k- nearest neighbors)
- Decision Trees
- Support Vector Machine

Rule- based classification

Advantages:

- Supervised learning allows collecting data and produces data output from previous experiences.
- Helps to optimize performance criteria with the help of experience.
- Supervised machine learning helps to solve various types of real-world computation problems.

Disadvantages:

- Classifying big data can be challenging.
- Training for supervised learning needs a lot of computation time. So, it requires a lot of time.

Unsupervised Learning

- Unsupervised learning is the training of a machine using information that is neither classified nor labeled and allowing the algorithm to act on that information without guidance.
- Here the task of the machine is to group unsorted information according to similarities, patterns, and differences without any prior training of data.
- Unlike supervised learning, no teacher is provided, which means no training will be given to the machine.
- Therefore the **machine is restricted to find the hidden structure in unlabeled data by itself.**

Unsupervised learning is classified into two categories of algorithms:

- **Clustering**: A clustering problem is where you want to discover the inherent groupings in the data, such as grouping customers by purchasing

behavior.

- **Association**: An association rule learning problem is where you want to discover rules that describe large portions of your data, such as people that buy X also tend to buy Y.

Clustering Types:

1. Hierarchical clustering
2. K-means clustering
3. Principal Component Analysis
4. Singular Value Decomposition
5. Independent Component Analysis

Supervised vs Unsupervised MachineLearning

Parameters	Supervised	Unsupervised
Input Data	**Algorithms are trained using labeled data.**	**Algorithms are used against data that is not labeled**
Computational Complexity	**Simpler method**	**Computationally complex**
Accuracy	**Highly accurate**	**Less accurate**

Decision Tree Induction

- Decision tree induction is the process of automatically building a decision tree model from a given dataset.

- A decision tree is a flowchart-like tree structure, where each internal node(non leaf node) denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node(or terminal node) holds a class label.
- **The topmost node in a tree is the root node.**
- Internal nodes are denoted by rectangles, and leaf nodes are denoted by ovals.
- Some decision tree algorithms produce only binary trees (where each internal node branches to exactly two other nodes), whereas others can produce nonbinary trees.

How are decision trees used for classification?

- Given a tuple, X, for which the associated class label is unknown, the attribute values of the tuple are tested against the decision tree.
- **A path is traced from the root to a leaf node, which holds the class prediction for that tuple.**
- Decision trees can easily be converted to classification rules.
- In general, decision tree classifiers have good accuracy. However, successful use may depend on the data at hand.
- Decision tree induction algorithms have been used for classification in many application areas such as medicine, manufacturing and production, financial analysis, astronomy, and molecular biology. Decision trees are the basis of several commercial rule induction systems.

Decision tree generation consists of two phases:

1. **Tree construction**
2. **Tree pruning**

- During **tree construction**, attribute selection measures are used to select the attribute that best partitions the tuples into distinct classes.

When decision trees are built,many of the branches may reflect noise or outliers in the training data.

- **Tree pruning** attempts to identify and remove such branches, with the goal of improving classification accuracy on unseen data.

Decision tree–based learning algorithms

- Iterative Dichotomiser (ID3)
- C4.5(successor of ID3)
- CART(Classification and Regression Tree)
- CHAID (Chi-square automatic interaction detection Performs multi-level splits when computing classification trees)
- MARS→(multivariate adaptive regression splines)

ID3, C4.5, and CART adopt a greedy (i.e., non backtracking) approach in which decision trees are constructed in a top-down recursive divide-and-conquer manner. Most algorithms for decision tree induction also follow a top-down approach, which starts with a training set of tuples and their associated class labels. The training set is recursively partitioned into smaller subsets as the tree is being built.

Decision Tree Creation

1. Start with the entire dataset at the root node
2. Select the **best attribute** to split the data (using measures like **Information Gain, Gain Ratio, or Gini Index**)
3. Create branches for each possible value of the attribute
4. Repeat the process for each branch
5. Stop when the data are pure or no attributes remain

Attribute Selection Measure

- ➢ Attribute Selection Measure (ASM) is a criterion used in decision tree induction to choose the best attribute for splitting the data at a node.

- ➢ **It decides which feature (attribute) should be used next to divide the dataset so that the resulting subsets are as pure or homogeneous as possible.**

- ➢ **The goal of an attribute selection measure is to:**
  - Reduce data impurity
  - Improve classification accuracy
  - Build a smaller and more efficient decision tree
- ➢ **There are three popular attribute selection measures— Information Gain, Gain ratio, and Gini index.**

Information Gain (IG)

- Information gain is a measure used to determine which feature should be used to split the data at each internal node of the decision tree. It is calculated using entropy.
- It is used in ID3.

$$IG(S, A) = Entropy(S) - \sum_{v \in A} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

Where:

- S = training dataset at the node
- A = candidate attribute
- S_v = subset after splitting on value v

Entropy

Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. In a decision tree, the goal is to decrease the entropy of the dataset by creating more pure subsets of data. Since entropy is a measure of impurity, by decreasing the entropy, we are increasing the purity of the data.

Shannon's entropy is a **measure of uncertainty or randomness** in a

dataset, originally proposed by **Claude Shannon** in information theory and widely used in **decision tree learning**.

- Consider a dataset with N classes. The entropy may be calculated using the formula below:

$$E = -\sum_{i=1}^{N} p_i log_2 p_i$$

  - **Pi** is the probability of randomly selecting an example in class i. The logarithm of fractions gives a negative value, and hence a '-' sign is used in the entropy formula to negate these negative values.

  - Let's have a dataset made up of three colours; red, purple, and yellow. our equation becomes:

$$E = -(p_r log_2 p_r + p_p log_2 p_p + p_y log_2 p_y)$$

Working of IG

1. Calculate entropy of the current dataset (root node)
2. Split the data using an attribute
3. Calculate weighted entropy of each subset
4. Subtract it from the original entropy
5. Attribute with **maximum Information Gain** is selected

Information Gain: Steps

The information gain is based on the decrease in entropy after a dataset is split on an attribute. Constructing a decision tree is all about finding attributes that return the

highest information gain (i.e., the most homogeneous branches).

Step1: Calculate entropy of the target.

Step 2: The dataset is then split on the different attributes. The entropy for each branch is calculated. Then it is added proportionally, to get total entropy for the split. The resulting entropy is subtracted from the entropy before the split. The result is the Information Gain, or decrease in entropy.

Step 3: Choose the attribute with the largest information gain as the decision node,divide the dataset by its branches and repeat the same process on every branch.

Step 4a: A branch with entropy of 0 is a leaf node.

Step 4b: A branch with entropy more than 0 needs further splitting.

Step 5: The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified.

Attribute with maximum Information Gain is selected

Gain Ratio

- **Gain Ratio is the ratio of Information Gain to Split Information and selects attributes that produce balanced and informative splits.**

$$\text{Gain Ratio}(A) = \frac{\text{Information Gain}(A)}{\text{Split Information}(A)}$$

Where,

$$\text{Split Information}(A) = -\sum_{v} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

Meaning of symbols

- **S**: total dataset
- **A**: attribute considered for split
- **v**: a value of attribute A
- **S_v**: subset of S where A = v

The attribute with the maximum gain ratio is selected as the splitting attribute.

Gini Index

- The Gini index is used in CART.
- The Gini index measures the impurity of D, a data partition or set of training tuples, as:

$$\text{Gini}(D)=1-\sum p2$$

Mathematically, The Gini Index is represented by

$$Gini\ impurity\ =\ 1\ -\ \Sigma(p(i)^2)$$

Another commonly used formula is:

$$Gini\ impurity\ =\ 1\ -\ \Sigma\left(p(i)\ *\ \left(1-p(i)\right)\right)$$

- Where p is the probability that a tuple in D belongs to class Ci and is estimated by |Ci, D|/|D|.
- The sum is computed over m classes.

The algorithm chooses the attribute / feature with the least Gini index as the root node.

Avoid Overfitting in Classification

- Overfitting occurs when a model learns the training data too well, capturing noise and random fluctuations in the data instead of the underlying patterns. As a result, an overfitted model performs well on the training set but fails to generalize to new, unseen data.

- The generated tree may overfit the training data – Too many branches, some may reflect anomalies due to noise or outliers – Result is in poor accuracy for unseen samples.

Tree Pruning

- When a decision tree is built, many of the branches will reflect anomalies in the training data due to noise or outliers.
- Tree Pruning methods address this problem of overfitting the data. Such methods typically use statistical measures to remove the least-reliable branches.
- Pruned trees tend to be smaller and less complex and, thus, easier to comprehend. They are usually faster and better at correctly classifying independent test data (i.e., of previously unseen tuples) than unpruned trees.

- There are two common approaches to tree pruning: **pre- pruning and post- pruning.**

Two approaches to avoid overfitting:

– **Pre- Pruning:** Halt tree construction early—do not split a node if this would result in the goodness measure falling below a threshold ▪ Difficult to choose an appropriate threshold.

– **Post- Pruning:** Remove branches from a "fully grown" tree—get a sequence of progressively pruned trees ▪ Use a set of data different from the training data to decide which is the "best pruned tree".

ID3 Algorithm

Steps in ID3 algorithm

The ID3 algorithm builds decision trees using a top-down greedy search approach through the space of possible branches with no backtracking. A greedy algorithm,as the name suggests, always makes the choice that seems to be the best at that moment.

- It begins with the original set S as the root node.
- On each iteration of the algorithm, it iterates through the very unused attribute of the set S and calculates Entropy(H) and **Information gain(IG)** of this attribute.
- It then selects the attribute which has the smallest Entropy or Largest Information gain.
- The set S is then split by the selected attribute to produce a subset of the data.
- The algorithm continues to recur on each subset, considering only attributes never selected before.

ID3 Algorithm : Example

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Step 1:

Entropy of entire Dataset , H(S)

= - p(yes) * log2(p(yes)) - p(no) * log2(p(no))

= -9/14 log2(9/14) - 5/14 log2(5/14)

= 0.94

Step 2:

First Attribute - Outlook

Categorical values - sunny, overcast and rain

H(Outlook=sunny) = -(2/5)*log2(2/5)-(3/5)*log2(3/5) =0.971

H(Outlook=rain) = -(3/5)*log2(3/5)-(2/5)*log2(2/5) =0.971

H(Outlook=overcast) = = 0

Average Entropy Information for Outlook -

I(Outlook) = p(sunny) * H(Outlook=sunny) + p(rain) * H(Outlook=rain) + p(overcast) * H(Outlook=overcast)

= (5/14)*0.971 + (5/14)*0.971 + (4/14)*0

= 0.693

Information Gain(Outlook) = H(S) - I(Outlook)

= 0.94 - 0.693

= 0.247

Step 3:

Second Attribute - Temperature

Categorical values - hot, mild, cool

H(Temperature=hot) = -(2/4)*log2(2/4)-(2/4)*log2(2/4) = 1

H(Temperature=cool) = -(3/4)*log2(3/4)-(1/4)*log2(1/4) = 0.811

H(Temperature=mild) = -(4/6)*log2(4/6)-(2/6)*log2(2/6) = 0.9179

Average Entropy Information for Temperature -

I(Temperature) = p(hot)*H(Temperature=hot) + p(mild)*H(Temperature=mild) + p(cool)*H(Temperature=cool)

= (4/14)*1 + (6/14)*0.9179 + (4/14)*0.811

= 0.9108

Information Gain = H(S) - I(Temperature)

= 0.94 - 0.9108

= 0.0292

Step 4:

Third Attribute - Humidity

Categorical values - high, normal

H(Humidity=high) = -(3/7)*log2(3/7)-(4/7)*log2(4/7) = 0.983

H(Humidity=normal) = -(6/7)*log2(6/7)-(1/7)*log2(1/7) = 0.591

Average Entropy Information for Humidity -

I(Humidity) = p(high)*H(Humidity=high) + p(normal)*H(Humidity=normal)

= (7/14)*0.983 + (7/14)*0.591

= 0.787

Information Gain = H(S) - I(Humidity)

= 0.94 - 0.787

= 0.153

Step 5:

Fourth Attribute - Wind

Categorical values - weak, strong

H(Wind=weak) = -(6/8)*log2(6/8)-(2/8)*log2(2/8) = 0.811

H(Wind=strong) = -(3/6)*log2(3/6)-(3/6)*log2(3/6) = 1

Average Entropy Information for Wind -

I(Wind) = p(weak)*H(Wind=weak) + p(strong)*H(Wind=strong)

= (8/14)*0.811 + (6/14)*1

= 0.892

Information Gain = H(S) - I(Wind)

= 0.94 - 0.892

= 0.048

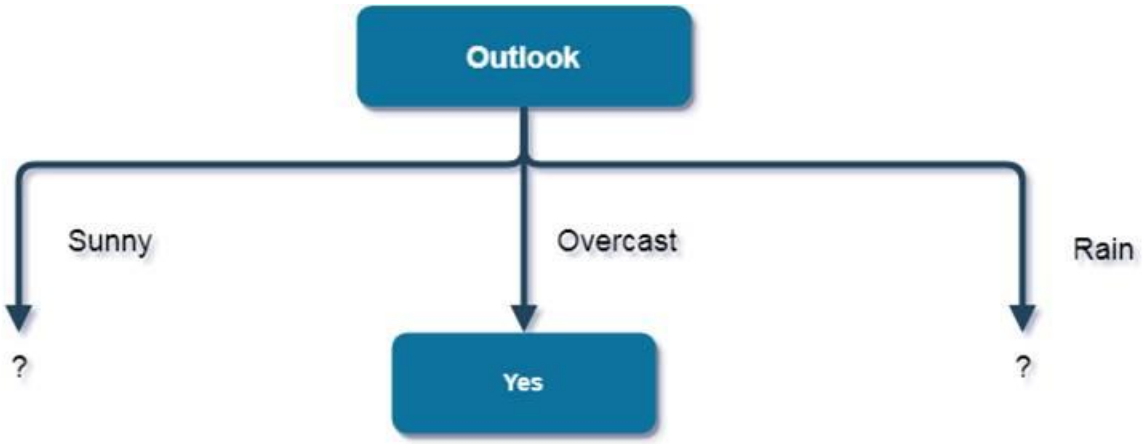
The Outlook has the greatest Information Gain. So, it can be used for the first split of the dataset.

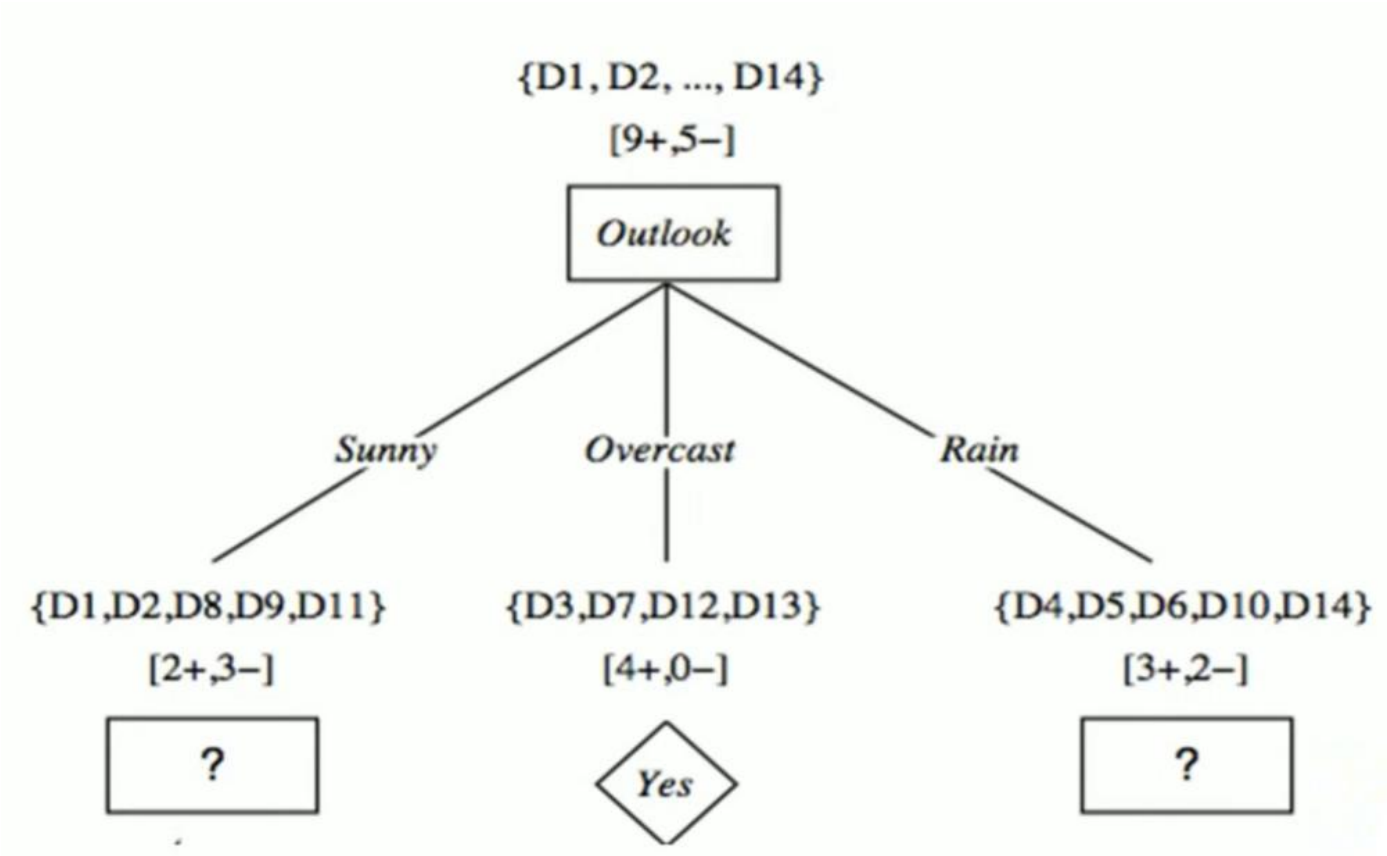
$$Gain(S, Outlook) = 0.2464$$

$$Gain(S, Temp) = 0.0289$$

$$Gain(S, Humidity) = 0.1516$$

$$Gain(S, Wind) = 0.0478$$





Step 6:

Finding the best attribute for splitting the data with Outlook=Sunny values{ Dataset rows = [1, 2, 8, 9, 11]}.

Complete entropy of Sunny is -

$$H(S) = - p(yes) * \log2(p(yes)) - p(no) * \log2(p(no))$$

$$= - (2/5) * \log2(2/5) - (3/5) * \log2(3/5)$$

$$= 0.971$$

Step 7:

First Attribute - Temperature

Categorical values - hot, mild, cool

H(Sunny, Temperature=hot) = -(0/2)log2(0/2)-2/2*log2(2/2) = 0

H(Sunny, Temperature=cool) = -(1/1)log2(1/1)- 0/1 log2(0/1) = 0

H(Sunny, Temperature=mild) = -(1/2)*log2(1/2)-(1/2)*log2(1/2) = 1

Average Entropy Information for Temperature -

I(Sunny, Temperature) = p(Sunny, hot)*H(Sunny, Temperature=hot) + p(Sunny, mild)*H(Sunny, Temperature=mild) + p(Sunny, cool)*H(Sunny, Temperature=cool)

= (2/5)*0 + (1/5)*0 + (2/5)*1

= 0.4

Information Gain = H(Sunny) - I(Sunny, Temperature)

= 0.971 - 0.4

= 0.571

Step 8:

Second Attribute - Humidity

Categorical values - high, normal

H(Sunny, Humidity=high) = - 0 - (3/3)*log(3/3) = 0

H(Sunny, Humidity=normal) = -(2/2)*log(2/2)-0 = 0

Average Entropy Information for Humidity -

I(Sunny, Humidity) = p(Sunny, high)*H(Sunny, Humidity=high) + p(Sunny, normal)*H(Sunny, Humidity=normal)

= (3/5)*0 + (2/5)*0

= 0

Information Gain = H(Sunny) - I(Sunny, Humidity)

= 0.971 - 0

= 0.971

Step 9:

Third Attribute - Wind

Categorical values - weak, strong

H(Sunny, Wind=weak) = -(1/3)*log(1/3)-(2/3)*log(2/3) = 0.918

H(Sunny, Wind=strong) = -(1/2)*log(1/2)-(1/2)*log(1/2) = 1

Average Entropy Information for Wind -

I(Sunny, Wind) = p(Sunny, weak)*H(Sunny, Wind=weak) + p(Sunny, strong)*H(Sunny, Wind=strong)

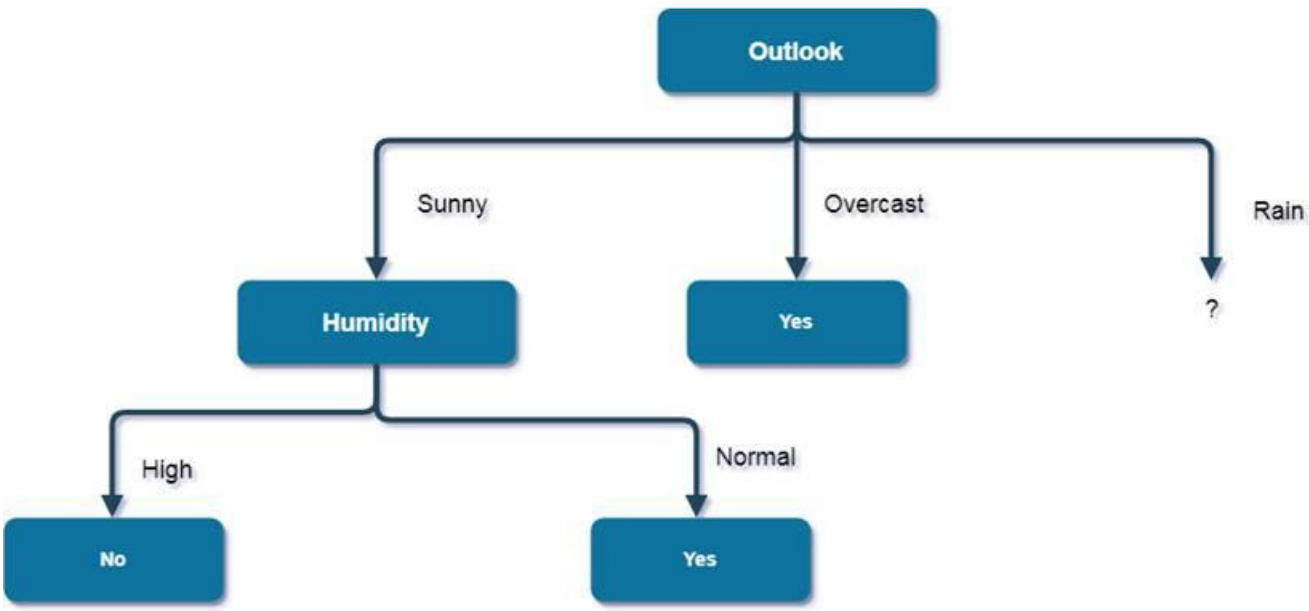
= (3/5)*0.918 + (2/5)*1

= 0.9508

Information Gain = H(Sunny) - I(Sunny, Wind)

= 0.971 - 0.9508

= 0.0202



Step 10:

Finding the best attribute for splitting the data with Outlook=Rainy values{ Dataset rows = [4, 5, 6, 10, 14]}.

Complete entropy of Rain is -

H(S) = - p(yes) * log2(p(yes)) - p(no) * log2(p(no))

= - (3/5) * log(3/5) - (2/5) * log(2/5)

= 0.971

First Attribute - Temperature

Categorical values - mild, cool

H(Rain, Temperature=cool) = -(1/2)*log(1/2)- (1/2)*log(1/2) = 1

H(Rain, Temperature=mild) = -(2/3)*log(2/3)-(1/3)*log(1/3) = 0.918

Average Entropy Information for Temperature -

I(Rain, Temperature) = p(Rain, mild)*H(Rain, Temperature=mild) + p(Rain, cool)*H(Rain, Temperature=cool)

= (2/5)*1 + (3/5)*0.918

= 0.9508

Information Gain = H(Rain) - I(Rain, Temperature)

= 0.971 - 0.9508

= 0.0202

Second Attribute - Wind

Categorical values - weak, strong

H(Wind=weak) = -(3/3)*log(3/3)-0 = 0

H(Wind=strong) = 0-(2/2)*log(2/2) = 0

Average Entropy Information for Wind -

I(Wind) = p(Rain, weak)*H(Rain, Wind=weak) + p(Rain, strong)*H(Rain, Wind=strong)

= (3/5)*0 + (2/5)*0

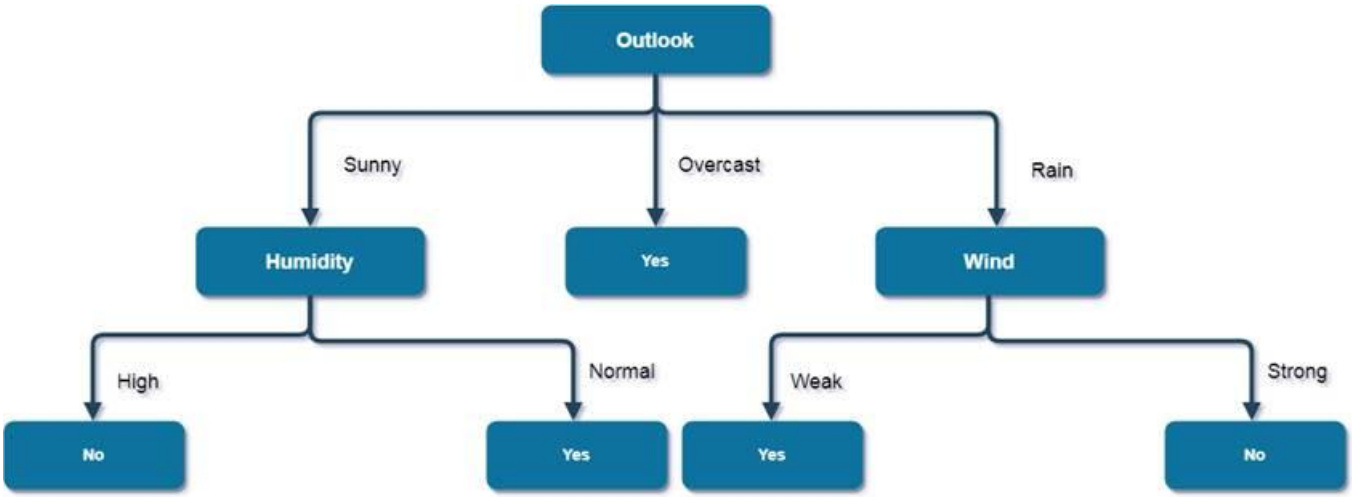
= 0

Information Gain = H(Rain) - I(Rain, Wind)

= 0.971 - 0

= 0.971

Here, the attribute with maximum information gain is Wind. So, the decision tree built so far -



Naive Bayes classifiers

Naïve Bayes classification is a **probabilistic supervised learning**

algorithm based on **Bayes' Theorem**, which assumes that **all features are conditionally independent given the class** (the "naïve" assumption).

❖ Naïve Bayes is not a single algorithm It is a family of algorithms (Gaussian, Multinomial, Bernoulli, etc.). All of them share the same core principle: they apply Bayes' theorem with an independence assumption.

Baye's Theorem

Bayes' theorem, named after 18th-century British mathematician Thomas Bayes, is a mathematical formula for determining conditional probability.

❖ Conditional probability is the likelihood of an outcome occurring, based on a previous outcome occurring.

❖ Bayes' theorem provides a way to revise existing predictions or theories (update probabilities) given new or additional evidence.

❖ In finance, Bayes' theorem can be used to rate the risk of lending money to potential borrowers.

Bayes' theorem relies on incorporating prior probability distributions in order to generate posterior probabilities.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \text{ if } P(B) \neq 0$$

❖ ***Prior probability***, in Bayesian statistical inference, is the probability of an event before new data is collected.

-This is the best rational assessment of the probability of an outcome based on the current knowledge before an experiment is performed.

❖ ***Posterior probability*** is the revised probability of an event occurring after taking into consideration new information.

-Posterior probability is calculated by updating the prior probability by using Bayes' theorem. In statistical terms, the posterior probability is the probability of event A occurring given that event B has occurred.

Fundamental Naive Bayes assumption

- Naïve Bayes uses Bayes' theorem to train the model.
- During training, it counts the occurrences of features for each class and assumes that all features are independent given the class.
- Finally, classification is done using the equation:

$$P(Y \mid X) \propto P(Y) \prod_{i=1}^{n} P(x_i \mid Y)$$

X – Input data / feature vector

Y – Class label

$P(Y)$ – Prior probability of the class

x_i – Individual feature (i = 1, 2, ..., n)

$P(x_i \mid Y)$ – Likelihood of a feature given the class

$\prod$ – Product operator (multiplication of probabilities)

$P(Y \mid X)$ – Posterior probability (final predicted probability)

❖ Is that each feature makes an:• independent •equal contribution to the outcome.

❖ We assume that no pair of features are dependent.

-For example, the temperature being 'Hot' has nothing to do with the humidity or the outlook being 'Rainy' has no effect on the winds. Hence, the features are assumed to be independent.

❖ Secondly, each feature is given the same weight(or importance).

-For example, knowing only temperature and humidity alone can't predict the outcome accurately. None of the attributes is irrelevant and assumed to be contributing equally to the outcome.

❖ ***Note: The assumptions made by Naive Bayes are not generally***

correct in real- world situations

Why Naïve Bayes is good

- Very **simple and fast**
- Works well with **large datasets**
- Excellent for **text classification** (spam detection, sentiment analysis)
- Performs well even when data is **limited**
- Easy to understand and explain (good for exams)

Why it is not always good

- Assumes features are **independent** (often not true)
- Can perform poorly when features are **strongly related**
- Less accurate for complex patterns

When it works best

- Text data (emails, documents, reviews)
- High-dimensional data (many features)
- Problems where **speed matters more than perfect accuracy**

When to avoid

- When features strongly depend on each other
- When relationships are highly complex

Algorithm

1. **Identify the classes**

   Determine all possible class labels $Y = \{y_1, y_2, \ldots, y_k\}$.

2. **Compute prior probabilities**

   Calculate the prior probability for each class:

$$P(y_j) = \frac{\text{Number of instances in } y_j}{\text{Total instances}}$$

3. **Compute likelihoods**

   For each feature x_i and class y_j, compute:

$$P(x_i \mid y_j)$$

   using frequency counts (or probability distribution for continuous data).

4. **Apply the Naïve independence assumption**

   Assume all features are conditionally independent given the class.

5. **Compute posterior probability**

   For a new instance $X = (x_1, x_2, \ldots, x_n)$, compute for each class:

$$P(y_j \mid X) \propto P(y_j) \prod_{i=1}^{n} P(x_i \mid y_j)$$

6. **Select the class with maximum probability**

   Assign the class with the highest posterior probability:

$$\hat{y} = \arg\max_{y_j} P(y_j \mid X)$$

Example:

Day	*Outlook*	*Temperature*	*Humidity*	*Wind*	*PlayTennis*
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Answer:

Step - 1

$$P(PlayTennis = yes) = 9/14 = .64$$

$$P(PlayTennis = no) = 5/14 = .36$$

Outlook	Y	N
sunny	2/9	3/5
overcast	4/9	0
rain	3/9	2/5
Tempreature		
hot	2/9	2/5
mild	4/9	2/5
cool	3/9	1/5

Humidity	Y	N
high	3/9	4/5
normal	6/9	1/5
Windy		
Strong	3/9	3/5
Weak	6/9	2/5

$$v_{NB} = \underset{v_j \in \{yes,no\}}{\operatorname{argmax}} P(v_j) \prod_i P(a_i|v_j)$$

$$= \underset{v_j \in \{yes,no\}}{\operatorname{argmax}} P(v_j) \quad P(Outlook = sunny|v_j) P(Temperature = cool|v_j)$$

$$\cdot P(Humidity = high|v_j) P(Wind = strong|v_j)$$

$$v_{NB}(yes) = P(yes)\ P(sunny|yes)\ P(cool|yes)\ P(high|yes)\ P(strong|yes) = .0053$$

$$v_{NB}(no) = P(no)\ P(sunny|no)\ P(cool|no)\ P(high|no)\ P(strong|no) = .0206$$

$$v_{NB}(yes) = \frac{v_{NB}(yes)}{v_{NB}(yes)+v_{NB}(no)} = 0.205 \qquad v_{NB}(no) = \frac{v_{NB}(no)}{v_{NB}(yes)+v_{NB}(no)} = 0.795$$

Since the V_{NB}(no) > V_{NB}(yes), the output will be No, that is can't go for playing Tennis

Example: 2

1. Consider the following hypothetical data concerning student characteristics and whether or not each student should be hired.

Name	GPA	Effort	Hirable
Sarah	poor	lots	Yes
Dana	average	some	No
Alex	average	some	No
Annie	average	lots	Yes
Emily	excellent	lots	Yes
Pete	excellent	lots	No
John	excellent	lots	No
Kathy	poor	some	No

Use a Naive Bayes classifier to determine whether or not someone with excellent attendance, poor GPA, and lots of effort should be hired.

Answer:

- $V_{NB}(Yes) = 0.125$
- $V_{NB}(No) = 0.05$

$$0.125 > 0.05$$

Final Decision

The person SHOULD be hired (Yes).

Rule-Based Classifier

Rule-based classifiers are just another type of classifier which makes the class decision by using various "if..else" rules. These rules are easily interpretable and thus these classifiers are generally used to generate descriptive models.

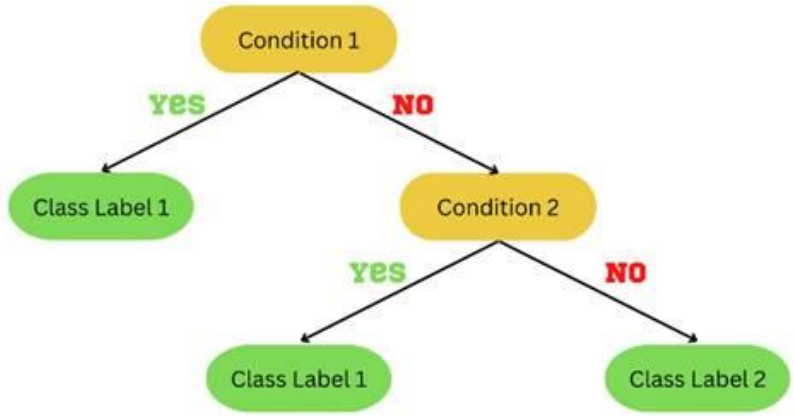
"IF condition THEN conclusion."

IF-THEN Rule

To define the IF-THEN rule, we can split it into two parts:

- **Rule Antecedent:** This is the "if condition" part of the rule. This part is present in the LHS (Left Hand Side). The antecedent can have one or more attributes as conditions, with logic AND operator.

- **Rule Consequent:** This is present in the rule's RHS (Right Hand Side). The rule consequent consists of the class prediction.



Example:

R1: IF climate= cool AND humidity = yes

THEN raining = true

Assessment of Rule

In rule-based classification in data mining, there are two factors based on which we can access the rules. These are:

- **Coverage of Rule:** The fraction of the records which satisfy the antecedent conditions of a particular rule is called the coverage of that rule.
  We can calculate this by dividing the number of records satisfying the rule(n1) by the total number of records(n).
  Coverage(R) = n1/n
- **Accuracy of a rule:** The fraction of the records that satisfy the antecedent conditions and meet the consequent values of a rule is called the accuracy of that rule.
  We can calculate this by dividing the number of records satisfying the consequent values(n2) by the number of records satisfying the rule(n1).
  Accuracy(R) = n2/n1

Example 1:

R1 : IF Status=Single THEN Class=No

Tid	Refund	Marital Status	Taxable Income	Class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Coverage = 4/10 = 40%

Accuracy = 2/4 = 50%

Example 2 :

Person	Age	Income	Purchase
1	Young	Low	No
2	Adult	High	Yes
3	Adult	Medium	No

R1 : IF Age=Young and Income=Low THEN Purchase=Nob

Find the coverage and accuracy of the above rule.

Coverage = $1/3$ =0.333 = 33%

Accuracy =1/1= 1=100 %

Properties of Rule-Based Classifiers

There are two significant properties of rule-based classification in data mining. They are:

- **Rules may not be mutually exclusive**
- **Rules may not be exhaustive**

Rules may not be mutually exclusive in nature

- If a rule R1 is satisfied by a tuple X, then the rule is said to be **triggered.**
- If **R1 is the only rule satisfied**, then the rule **fires** by returning the class **prediction for X.**
- Triggering does not always mean firing because there may be more than one rule that is satisfied.
- Many different rules are generated for the dataset, so it is possible and likely that many of them satisfy the same data record. **This condition makes the rules not mutually exclusive.**
- Since the rules are not mutually exclusive, we cannot decide on classes that cover different parts of data on different rules.

Multiple rules can match the same record, causing confusion. To fix this, we use special methods to decide which rule should finally be used.

- We have two ways:
  - Size Ordering
  - Rule Ordering

Size Ordering

The size ordering scheme assigns the highest priority to the triggering rules that have the toughest requirements, **where toughness is measured by the rule antecedent size**. That is, the triggering rule with the most attribute tests is fired.

- When **many rules match the same data**, we give priority to the **strongest rule.**
- The **strongest rule** is the one with **more conditions** (more "IF" parts).
- A rule with more conditions is more **specific and strict.**
- So, **the rule with the largest number of conditions fires.**

Example:

- Rule 1: IF GPA = excellent → Hire
- Rule 2: IF GPA = excellent AND Effort = lots → Hire

  Rule 2 fires because it has **more conditions.**

Rule Ordering

- The rule ordering scheme prioritizes the rules beforehand. The ordering may be **class-based** or **rule-based**.
- **With class-based ordering,** the classes are sorted in order of decreasing "importance" such as by decreasing order of prevalence. That is, all the rules for the most prevalent (or most frequent) class come first, the rules for the next prevalent class come next, and so on. Alternatively, they may be sorted based on the misclassification cost per class. Within each class, the rules are not ordered—they don't have to be because they all predict the same class (and so there can be no class conflict!). **So, in this, the set of rules remains unordered.**

- First, decide **which class is more important.**
- Classes may be ordered:
  - by how **frequent** they are in data, or
  - by **misclassification cost.**
- All rules predicting the **most important class** are placed first.
- Rules predicting the next important class come next.
- **Within the same class, rules are not ordered**, because they all predict the same result.

Since all rules in a group give the same class, **no conflict occurs.**

- **With rule-based ordering,** the rules are organized into one long priority list, according to some measure of rule quality, such as accuracy, coverage, size (number of attribute tests in the rule antecedent), or based on advice from domain experts. When rule ordering is used, the rule set is known as a **decision list**. With rule ordering, the triggering rule that appears earliest in the list has the highest priority, and so it gets to fire its class prediction. Any other rule that satisfies X is ignored. **Here it uses an ordered set of rules.**

  - Every rule is placed into **one long ordered list.**
  - Rules are ranked based on:
    - accuracy
    - coverage
    - number of conditions
    - expert knowledge
  - This ordered list is called a **decision list.**
  - When a data record matches:
    - **the first matching rule fires**
    - all other matching rules are **ignored**

  Here, **order matters**, and only **one rule is used.**

Rules may not be exhaustive in nature

- It is not a guarantee that the rule will cover all the data entries.
- Any of the rules may leave some data entries.
- This case, on its occurrence, will make the rules non-exhaustive.
- So, we have to solve this problem too.
- So, to solve this problem, we can make use of a **default class**. Using a default class, we can assign all the data entries not covered by any rules to the default class.
- Thus using the default class will solve the problem of non-exhaustively.

Rule Induction Using Sequential Covering Algorithm

- **Sequential Covering Algorithm** can be used to extract IF-THEN rules from the training data without constructing a decision tree.

- Some of the sequential covering algorithms are AQ, CN2, and RIPPER.
- As per the general strategy the rules are learned one at a time.
- For each time rules are learned, a tuple covered by the rule is removed and the process continues for the rest of the tuples. This is because the path to each leaf in a decision tree corresponds to a rule.
- Sequential covering algorithms are the most widely used approach to mining disjunctive sets of classification rules, and form the topic of this subsection. Note that in a newer alternative approach, classification rules can be generated using associative classification algorithms, which search for

attribute-value pairs that occur frequently in the data. These pairs may form association rules, which can be analyzed and used in classification.

Algorithm: Sequential Covering

Input:

- D, a data set of class-labeled tuples;
- Att_vals, the set of all attributes and their possible values.

Output:

A set of IF-THEN rules.

Method:

```
(1) Rule_set = {}; // initial set of rules learned is empty
(2) for each class c do
(3) repeat
(4) Rule = Learn_One_Rule(D,Att_vals,c);
(5) remove tuples covered by Rule from D;
(6) Rule_set = Rule_set + Rule; // add new rule to rule set
(7) until terminating condition;
(8) endfor
(9) return Rule_Set
```

- **If rules are not mutually exhaustive, how would the class be decided in case different rules with different consequent cover the record.**
- **There are two solutions to the above problem:**
  - Either rules can be ordered ,i.e. the class corresponding to the highest priority rule triggered is taken as the final class.
  - Otherwise, we can assign votes for each class depending on some of their weights, i.e., the rules remain unordered.

Working Steps

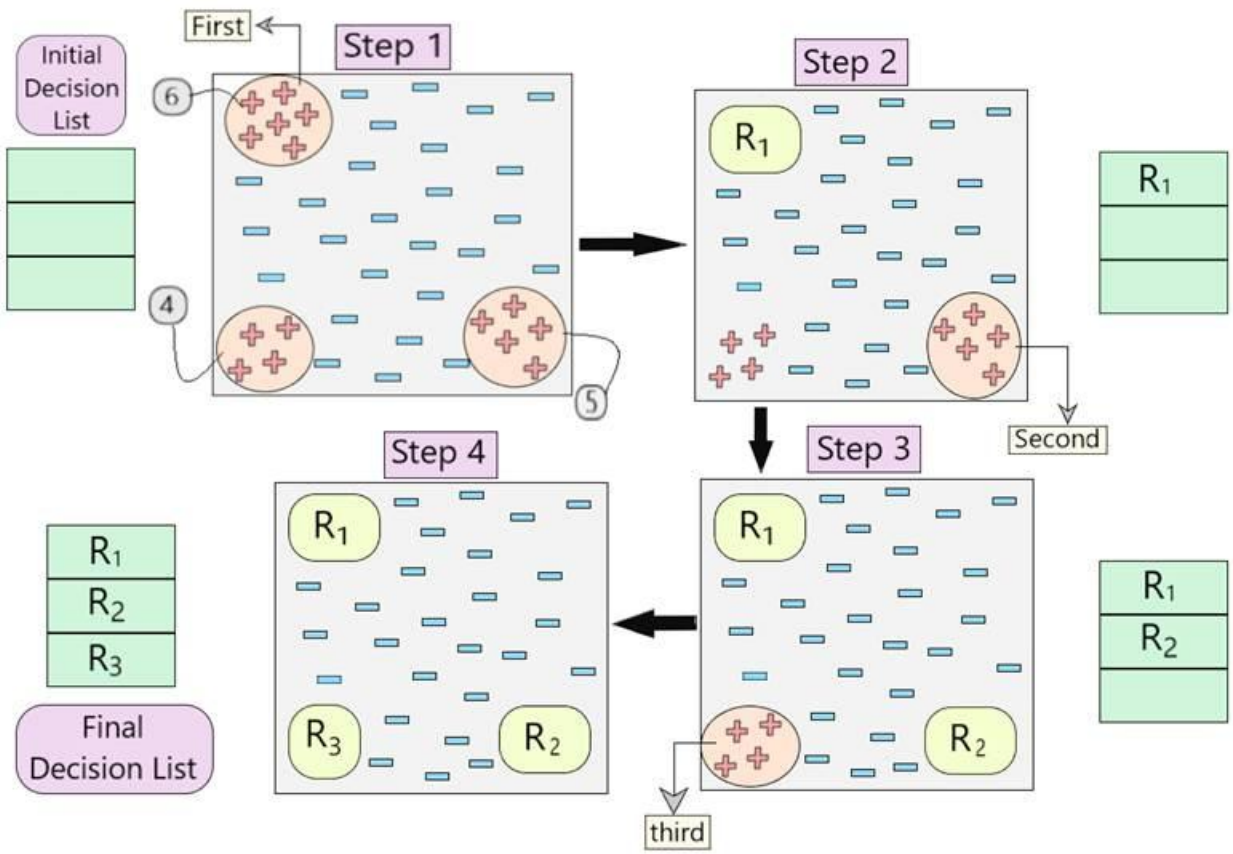
1. **Start with all training examples**

2. **Learn the best rule that covers many positive examples**

3. **Remove the covered examples**

4. **Repeat until no positive examples remain**

Below, is a visual representation describing the working of the algorithm.



- First, we created an empty decision list. During Step 1, we see that there are three sets of positive examples present in the dataset. So, as per the algorithm, we consider the one with maximum no of positive examples. (6, as shown in Step 1).

- Once we cover these 6 positive examples, we get our first rule R1, which is then pushed into the decision list and those positive examples are removed from the dataset. (as shown in Step 2).

- Now, we take the next majority of positive examples (5, as shown in Step 2) and follow the same process until we get rule R2. (Same for R3)

- In the end, we obtain our final decision list with all the desirable rules.

K-Nearest Neighbor(KNN) Algorithm

- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.

- The K-NN algorithm assumes the similarity between the new case/data and available cases and puts the new case into the category that is most similar to the available categories.

- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suited category by using K- NN algorithm.

- The K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

- K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data.

- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.

- The KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

- Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know whether it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

Implementing a KNN model

1. Load the data
2. Initialize the value of **k**
3. For getting the predicted class, iterate from 1 to total number of training data points

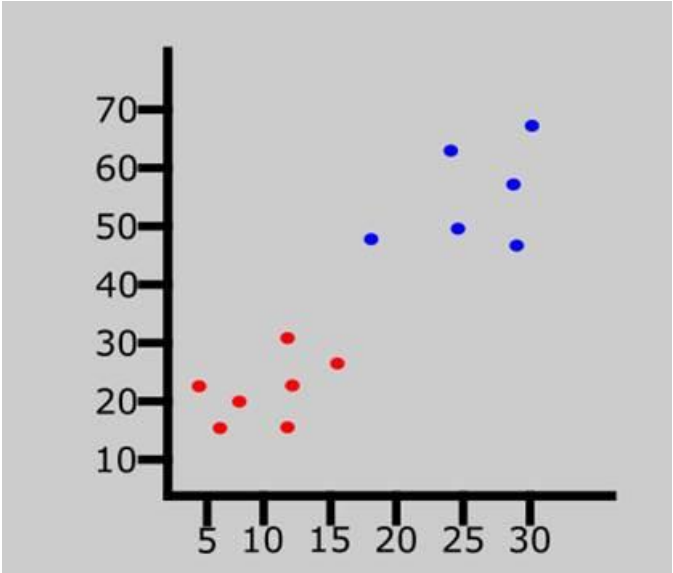
    - Calculate the distance between test data and each row of training dataset. Use Euclidean distance as our distance metric since it's the most popular method. The other distance functions or metrics that

can be used are Manhattan distance, Minkowski distance, Chebyshev, cosine, etc. If there are categorical variables, hamming distance can be used.

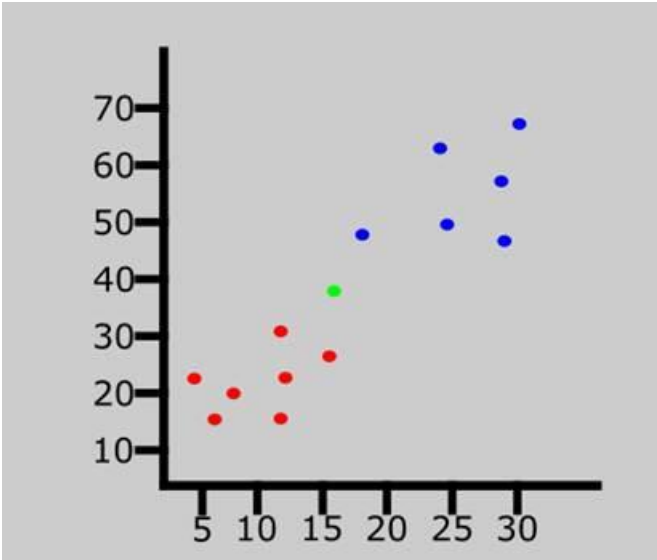
- Sort the calculated distances in ascending order based on distance values
- Get top k rows from the sorted array
- Get the most frequent class of these rows
- Return the predicted class (ie, frequent class).

Example:

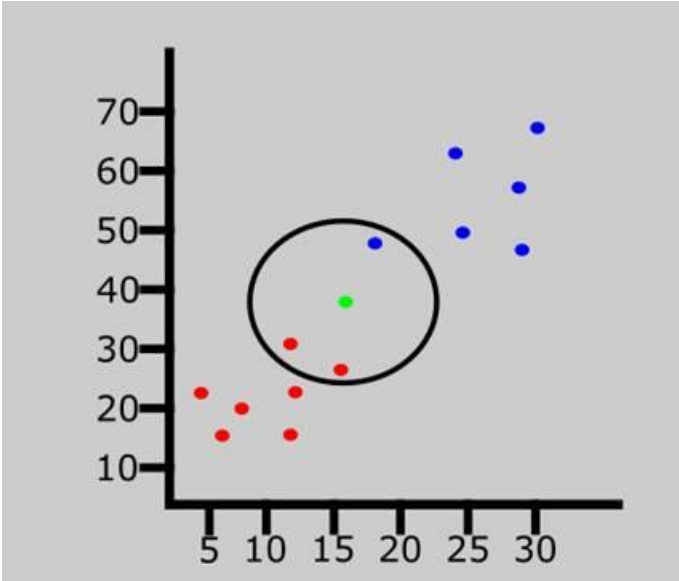
- Consider the diagram below: It is a data set consisting of two classes — red and blue.



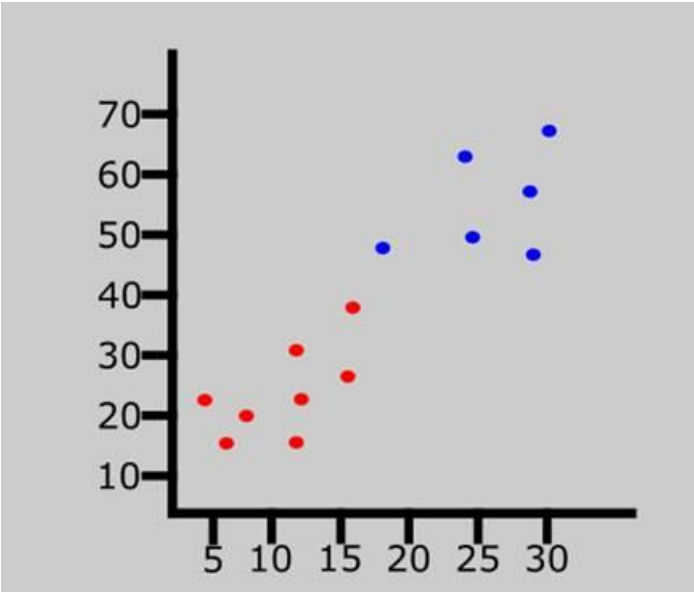
- A new data entry has been introduced to the data set. This is represented by the green point in the graph above.



- Assign a value to K which denotes the number of neighbors to consider before classifying the new data entry. Let's assume the value of K is 3.
- Since the value of K is 3, the algorithm will only consider the 3 nearest neighbors to the green point (new entry).



- Out of the 3 nearest neighbors in the diagram above, the majority class is red so the new entry will be assigned to that class.



Example:

Consider the following data set

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

Predict the class for the following

BRIGHTNESS	SATURATION	CLASS
20	35	?

Solution:

Calculate the distance from the new entry to other entries in the data set using the Euclidean distance formula.

Here's the formula: $\sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$

Where:

- X_2 = New entry's brightness (20).
- X_1= Existing entry's brightness.
- Y_2 = New entry's saturation (35).
- Y_1 = Existing entry's saturation.

Distance #1

For the first row, d1:

BRIGHTNESS	SATURATION	CLASS
40	20	Red

$$d1 = \sqrt{(20 - 40)^2 + (35 - 20)^2}$$
$$= \sqrt{400 + 225}$$
$$= \sqrt{625}$$
$$= 25$$

We now know the distance from the new data entry to the first entry in the table. Let's update the table.

BRIGHTNESS	SATURATION	CLASS	DISTANCE
40	20	Red	25
50	50	Blue	?
60	90	Blue	?
10	25	Red	?
70	70	Blue	?
60	10	Red	?
25	80	Blue	?

BRIGHTNESS	SATURATION	CLASS	DISTANCE
40	20	Red	25
50	50	Blue	33.54
60	90	Blue	68.01
10	25	Red	10
70	70	Blue	61.03
60	10	Red	47.17
25	80	Blue	45

Let's rearrange the distances in ascending order:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17
70	70	Blue	61.03
60	90	Blue	68.01

Since we chose 5 as the value of K, we'll only consider the first five rows. That is:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17

As you can see above, the majority class within the 5 nearest neighbors to the new entry is **Red**. Therefore, we'll classify the new entry as **Red**.